Relatório COMINDIR02

Particionamento e Recuperação com Publish/Subscribe (IPFS PubSub + TCP)

Documento de estudo: explica, passo a passo, **o que foi construído**, **o que é cada peça**, **como foi testado** (com os valores realmente medidos) e inclui **as perguntas que podem ser feitas na avaliação oral**, com respostas. Issue 15 do repositório SD-2026-2027. Código em `Sprints/E1-COMINDIR02/` (projetos `rep02-produtor`, `rep02-lider`, `rep02-replica`, e pasta `teste/` com o simulador e os cenários).

1. Resumo em 1 minuto

- **Problema herdado da COMINDIR01:** (1) uma réplica que perde um registo fica retida para sempre; (2) todas as réplicas guardam tudo; (3) há um único líder.
- **Solução A – Recuperação:** a réplica deteta a lacuna e, quando tem **mais de 10 registos retidos**, liga-se por **TCP** ao líder e pede o intervalo em falta. O líder diz onde pode ser contactado dentro de **cada** mensagem que publica.
- **Solução B – Particionamento:** os dados são divididos em **N = 3 partições** pela chave `floorMod(sensor.hashCode(), N)`. Cada partição tem o seu líder, o seu tópico, o seu porto TCP (7000+k) e a sua sequência `seq`. Cada partição existe em **R = 2** réplicas.
- **Propagação** continua a ser PubSub do IPFS (desacoplado). **Só a recuperação** usa comunicação direta (TCP), e por isso reintroduz acoplamento no espaço e no tempo.
- **Resultado dos testes:** em todos os cenários as réplicas convergiram para o ficheiro do líder, exceto nos casos que o mecanismo **não cobre por desenho** (perda dos últimos registos de uma série, líder inacessível, líder sem histórico). Testado contra um **daemon Kubo 0.43.1 real** (1 e 2 nós).

2. Glossário: o que é cada coisa

Termo	O que é	Onde aparece
PubSub (publish/subscribe)	Comunicação indireta: o emissor publica num **tópico**, quem subscreve recebe. Emissor e recetor não se conhecem (desacoplamento no **espaço**) nem têm de estar ativos ao mesmo tempo (no **tempo**, em parte). Entrega *best-effort*.	`IpfsPubSub`, tópicos `rep02-*`
IPFS / Kubo	Rede P2P; o **Kubo** é o daemon (`ipfs`). Aqui só se usa o PubSub dele, através da API HTTP em `127.0.0.1:5001`.	`IpfsPubSub.publish/subscribe`
Multibase	Codificação que o Kubo exige: base64url sem padding com prefixo u, aplicada ao nome do tópico e aos dados.	`IpfsPubSub.toMultibase`
Registo	Linha `seq;sensor;temperatura;instante`. É o que se guarda nos ficheiros e se envia.	`SensorRecord`
`seq` (número de sequência)	Atribuído **só pelo líder**, 1,2,3… sem saltos. Permite às réplicas detetar lacunas, duplicados e desordem.	`PartitionLeader`
Líder	Processo que recebe leituras (`IN`), atribui `seq`, escreve no seu ficheiro e publica `REG`. Um por partição.	`PartitionLeader`
Réplica	Processo que subscreve os tópicos `REG` das suas partições e guarda cópia do ficheiro do líder.	`PartitionReplica`
Produtor	Recebe leituras do utilizador, calcula a partição e publica `IN` no tópico de entrada correspondente. Não conhece líderes.	`Producer`
Mensagem `IN`	`IN;<sensor>;<temp>`: leitura ainda sem `seq`. Produtor → líder.	`Wire.in`
Mensagem `REG`	`REG;<endereço>;<porto>;<seq>;<sensor>;<temp>;<instante>`: registo definitivo + onde recuperar. Líder → réplicas.	`Wire.reg`
Retido	Registo que chegou com `seq` maior que `último+1`: guardado em memória (`TreeMap`) à espera dos anteriores.	`Part.retained`
`LIMITE_RETIDOS` = 10	Quando há **mais de 10** retidos, a réplica deixa de esperar (considera perdido) e recupera.	`Wire`
Recuperação a pedido do recetor	A réplica deteta o que falta e pede. Alternativa: **ACK positivo** (o emissor tem de conhecer os recetores, perdendo o desacoplamento).	`PartitionReplica.recover`
Partição / *sharding*	Subconjunto disjunto dos dados. Aqui: por sensor.	`Partitioner`

Termo	O que é	Onde aparece
Chave de partição	Campo que decide a partição: o identificador do sensor.	`Partitioner.of`
Fator de replicação R	Nº de réplicas por partição (R = 2).	atribuição 0,1 / 1,2 / 2,0
`floorMod`	Resto da divisão que é sempre ≥ 0. Com %, um `hashCode` negativo daria índice negativo (`-7 % 3 = -1`).	`Partitioner`
`ServerSocket` / `Socket`	TCP: o líder escuta no porto 7000+k (`accept()` bloqueia); a réplica liga-se (`connect`) e fala por linhas.	líder / réplica
`T_TCP_MS` = 5000	Tempo máximo de ligação (`connect`) e de espera por cada linha (`setSoTimeout`). Sem isto, `readLine` bloquearia para sempre.	`Wire`
Secção sincronizada	`synchronized` à volta de «atribuir `seq` + `append`» para que nenhuma outra thread veja um estado intermédio.	`PartitionLeader.onInput`

3. Arquitetura

Três tipos de processo, três canais. Tudo o que é propagação passa pelo PubSub; só a recuperação é TCP.

Canal	Mensagens	Quem envia	Quem recebe
tópico `rep02-in-p<k>`	`IN;sensor;temp`	produtor(es)	líder da partição k
tópico `rep02-p<k>`	`REG;endereço;porto;seq;sensor;temp;instante`	líder da partição k	réplicas da partição k
TCP `endereço:7000+k`	`PEDIDO de até` → registos → `FIM` (ou `ERRO motivo`)	réplica	líder da partição k

Réplica	Partições	Ficheiros
1	p0, p1	`replica-1-p0.txt`, `replica-1-p1.txt`
2	p1, p2	`replica-2-p1.txt`, `replica-2-p2.txt`
3	p2, p0	`replica-3-p2.txt`, `replica-3-p0.txt`

Cada partição está em exatamente 2 réplicas, por isso a falha de uma réplica não torna nenhuma partição indisponível. O líder da partição k escreve `lider-p<k>.txt`.

Fluxo de uma leitura

- 1. O utilizador escreve `S02 22.0` no produtor. `Partitioner.of("S02")` = 1 → publica `IN;S02;22.0` em `rep02-in-p1`.
- 2. O líder 1 (thread `entrada-p1`, bloqueada em `subscribe`) recebe, confirma que S02 pertence a p1 e, **numa secção sincronizada**, faz `seq+1` e `append` ao ficheiro.
- 3. **Depois** (fora do lock) publica `REG;127.0.0.1;7001;<seq>;S02;22.0;<instante>` em `rep02-p1`.
- 4. As réplicas 1 e 2 (as que têm p1) recebem, aprendem `127.0.0.1:7001` como endereço do líder e aplicam a regra: escrever, reter ou ignorar duplicado.

4. Passo a passo da implementação

4.0 Classes copiadas da COMINDIR01 (só mudou o pacote para `rep02`)

- `SensorRecord`: `seq`, `sensorId`, `temperature`, `timestamp`; `toLine()` / `fromLine()`.
- `RecordFile`: `append` (acrescenta linha) e `lastSeq()` (último `seq` do ficheiro, para retomar após reinício).
- `IpfsPubSub`: `publish(tópico, msg)` e `subscribe(tópico, callback)` sobre HTTP (`java.net.http.HttpClient`). O `subscribe` **não termina**: lê a resposta linha a linha.

4.1 Parte A, ponto 1: `RecordFile.range(from, to)`

Lê o ficheiro inteiro e devolve os registos com `from ≤ seq ≤ to`. É a **fonte da recuperação**: o líder responde com o que está no seu ficheiro. É `synchronized`, tal como `append`, por isso **nunca lê a meio de um** `append`.

P: Como cresce o custo de um pedido com o histórico? Que informação adicional evitaria ler tudo?

R: Linearmente, O(tamanho do ficheiro) por pedido, mesmo que o intervalo pedido seja pequeno (lê e faz *parse* de todas as linhas). Um **índice** `seq → posição (byte offset)` permitiria saltar diretamente para `from`. Fica fora do âmbito.

4.2 Ponto 2: `Wire`

Concentra constantes (`LIMITE_RETIDOS=10`, `T_TCP_MS=5000`, `PORTO_BASE=7000`), nomes dos tópicos (`topicIn(k)`, `topicData(k)`) e formato das mensagens (`in`, `reg`, `type`, `body`, `regParts`). Está nos três projetos para que **produtor, líder e réplica concordem no formato**. `regParts` usa `split(";",4)` para separar `endereço`, `porto` e o resto (que é a linha do registo, com os seus próprios `;`).

4.3 Ponto 3: `Producer`

- `Producer <N> [api]`. Lê comandos de stdin: `<sensor> <temp>`, `rajada <n> <s>`, `rajada-quente <n> <s>`, `sair`.
- Calcula `k = partitioner.of(sensor)` e publica em `Wire.topicIn(k)`; escreve `→ p<k>`.
- `rajada`: n leituras de `S01..S<s>`, temperaturas aleatórias 15–30 °C; imprime instantes do primeiro e último envio, duração e **quantas foram para cada partição**.
- `rajada-quente`: 80 % das leituras vêm de `S01`; as restantes de `S02..S<s>`.
- **Descoberta durante os testes:** o Kubo (no Windows) por vezes responde HTTP 500 «validation ignored» a duas publicações muito próximas. O produtor e o líder repetem a publicação (até 8 vezes, com 2 ms de pausa). O número de repetições aparece no fim da rajada (`repetições=`).

4.4 Ponto 4: `PartitionLeader <k> <N> [endereço] [api]`

Tem **duas atividades concorrentes** no mesmo processo:

Atividade	Thread	Onde bloqueia	Estado partilhado
Entrada	`entrada-p<k>` (nova)	`subscribe(...)` à espera de cada `IN`	escreve `seq` e o ficheiro
Recuperação	`main`	`accept()` à espera de réplica; `readLine()` à espera do pedido	só lê `seq` (via `lastSeq()`) e o ficheiro (`range`)

- **Estado partilhado:** o último `seq` atribuído, protegido por um `Object LOCK`. A atribuição do `seq` e o `append` estão **na mesma secção** `synchronized`; `lastSeq()` também usa o `LOCK`.
- **Validação:** mensagem inválida → `REJEITADO: <conteúdo>` e continua. Sensor de outra partição → `PARTIÇÃO ERRADA: <sensor> pertence a p<j>`.
- **Ordem «escrever localmente, depois propagar»:** garante que tudo o que as réplicas viram anunciado já está no ficheiro do líder, logo pode ser recuperado.
- **Servidor TCP:** atende **uma ligação de cada vez**. Lê `PEDIDO <de> <até>`; se mal formado ou `de>até` → `ERRO <motivo>`. Limita `até` ao último `seq` escrito, envia `range(de,até)` linha a linha e termina com `FIM`. Regista `RECUPERAÇÃO: <de>..<até> para <remoto> (<n> registos)`.
- Uma exceção numa ligação só termina essa ligação (`catch IOException` dentro do ciclo `while`).
- Abre o `ServerSocket` **antes** de aceitar leituras; se o porto estiver ocupado, escreve o erro e termina com código 1.
- O **endereço anunciado** é argumento (`127.0.0.1` por omissão) porque `InetAddress.getLocalHost()` devolve o endereço que a máquina conhece de si própria, que pode não ser alcançável pelas réplicas (NAT, várias interfaces, VPN).
- Opção de teste `-Dsemfim=true`: o líder fecha a ligação a meio da resposta (simula a falha 5 da tabela).

4.5 Ponto 5: `PartitionReplica <id> <partições> [portoApi]`

Por cada partição há um objeto `Part` com: ficheiro, último `seq` escrito (inicializado com `lastSeq()`), `TreeMap<Long,SensorRecord> retained`, endereço/porto do líder e contadores. Cada partição tem a sua **thread de subscrição**.

```
regra de aplicação (por partição)
REG com seq == último+1           -> escreve; depois escreve os retidos que ficaram em sequência (flush)
REG com seq <= último / já retido -> DUPLICADO: ignora
REG com seq >  último+1           -> retém (EM ESPERA)
   se retidos > LIMITE_RETIDOS    -> recover(): PEDIDO (último+1) .. (menor retido - 1)
recuperação falhou                -> mantém o que já escreveu; tenta de novo na mensagem seguinte
```

- **Aprende o líder** com cada `REG` (atualiza endereço/porto sempre): se o líder mudar de máquina, a próxima mensagem já aponta para o novo sítio.

- **Recuperação na thread da própria partição:** `connect(host,porto,T_TCP_MS)`, `setSoTimeout(T_TCP_MS)`, envia `PEDIDO`, lê até `FIM`, aplica cada registo com a mesma regra, depois `flush`. Qualquer `IOException` (ligação recusada, *timeout*, ligação fechada sem `FIM`, `ERRO`) → `RECUPERAÇÃO FALHOU: <motivo>`; o que já foi escrito mantém-se.
- **Enquanto recupera**, essa thread não lê o seu `pubsub/sub`; as mensagens da partição **ficam em fila** (buffers TCP/Kubo) e são lidas a seguir. Outras partições têm outras threads e não são afetadas.
- **Simulação de perda** só no PubSub: `-Dperda=<p>` descarta cada `REG` com probabilidade p; `-Ddescartar=<seq>` descarta só a 1.ª receção desse `seq`. Imprime `DESCARTADO (simulação): <seq>`.
- **Sincronização:** o estado de cada partição só é tocado pela sua thread de subscrição e pelo *idle watcher* (que imprime estatísticas); por isso cada `Part` é o seu próprio monitor (`synchronized(p)`). Não há estado partilhado entre partições.

4.6 Parte B, ponto 7: `Partitioner`

```
int particao = Math.floorMod(sensorId.hashCode(), n);
```

`String.hashCode()` está definido pela especificação da linguagem, logo é **igual em qualquer JVM** (produtor e líder concordam sem comunicar). Medido para `S01..S20` com N = 3: **7 / 7 / 6** sensores por partição. De N = 3 para N = 4, **13 dos 20 sensores (65 %) mudam de partição**.

4.7 Pontos 8 a 10

- **Produtor** passa a receber N e a escrever → `p<k>`; novo comando `rajada-quente`.
- **Líder** verifica a partição de cada `IN` (defesa contra produtores/líderes configurados com N diferente). A verificação **tem de estar no líder** porque é ele que escreve; o produtor pode estar mal configurado ou ser outro programa que publique diretamente.
- **Réplica** generalizada: `PartitionReplica 1 0,1, 2 1,2, 3 2,0`: uma thread e um estado por partição.

4.8 Ponto 11: `PartitionCheck <N> <pasta>`

Lê `lider-p*.txt` e `replica-*-p*.txt` e verifica, por partição: (1) todos os sensores do líder têm `of(sensor)==k`; (2) `seq` começa em 1, sem saltos nem repetições; (3) quantas réplicas (deve ser 2) e se cada uma está **IGUAL**, é **PREFIXO** (e quantos faltam) ou **DIVERGE**; (4) nº de registos e sensores distintos. No fim, total do sistema = soma das partições.

5. Como executar

```
# compilar (em cada projeto)
javac -d out src/rep02/*.java

# réplicas primeiro
java -cp out rep02.PartitionReplica 1 0,1
java -cp out rep02.PartitionReplica 2 1,2
java -cp out -Dperda=0.2 rep02.PartitionReplica 3 2,0
# depois os líderes
java -cp out rep02.PartitionLeader 0 3
java -cp out rep02.PartitionLeader 1 3
java -cp out rep02.PartitionLeader 2 3
# por fim o produtor
java -cp out rep02.Producer 3
> rajada 3000 20
> rajada-quente 3000 20

# verificação (ficheiros reunidos numa pasta)
java -cp out rep02.PartitionCheck 3 pasta/
# testar o líder à mão
nc 127.0.0.1 7000   ->   PEDIDO 1 5
# ver subscrições / portos
ipfs pubsub ls         netstat -an | findstr 700
```

Testes automáticos: `teste/Cenarios.java` lança os processos reais (JVMs separadas, pastas separadas) contra o Kubo e compara ficheiros. `teste/MockIpfs.java` é um nó IPFS simulado (só para quem não tenha Kubo). Exemplo: `java -Dreal=true -cp teste/out Cenarios <pasta> A-d B-b F-semfim`.

6. Testes realizados e valores medidos

Ambiente: Windows 11, Java 21, Kubo 0.43.1 real (nó 1 em :5001; nó 2 em :5002, ligado ao 1, usado no teste 6(f)). **Todos os processos na mesma máquina**, por isso os tempos medem sobretudo o Kubo e a CPU/disco local, não a rede. Os valores variam muito de execução para execução (a primeira rajada de cada série é por vezes 2–5× mais lenta); indicam-se as 3 execuções e a média.

6.1 Parte A (uma partição)

Teste	Resultado medido
(a) 5 leituras	Os 4 ficheiros (líder + 3 réplicas) **iguais** (5/5).
(b) réplica 2 parada: 3 leituras, volta, +2	Réplica 2 ficou com **5/10 (PREFIXO)**, **2 retidos** (seq 9 e 10), 0 recuperações. Leituras uma a uma: a recuperação foi desencadeada pela **leitura seq = 19** (11.º retido: 9..19). Pediu `6..8`, obteve 3 registos; fim: **19/19 IGUAL**.
(c) réplica 3 com `-Ddescartar=6`, só essa leitura	Ficheiro da réplica 3 ficou em **5/6 (PREFIXO, falta 1)** e **não converge sozinho**. Foram precisas **11 leituras adicionais** (seq 7..17) para haver 11 retidos; então pediu `6..6` e ficou **17/17 IGUAL**.
(e) réplica 3 nova (ficheiro apagado) com 200 registos no líder, depois `rajada 20`	**1 ligação TCP**, **200 registos via TCP** (`1..200`), ficheiro **220/220 IGUAL**; convergência **184 ms** após a 1.ª publicação da rajada.

Tabela de recuperação (`rajada 1000 10`, 1 partição, médias de 3 execuções; entre parênteses as 3 execuções)

Métrica	COMINDIR01 (1 nó)	COMINDIR02 sem perdas (1 nó)	COMINDIR02, réplica 3 com perda 0,2 (1 nó)	idem, 2 nós
Tempo de envio no produtor (ms)	2 916 (3142; 3021; 2584)	4 374 (7109; 2854; 3159)	10 937 (15627; 8942; 8241)	5 107 (4470; 5234; 5618)
Tempo até à convergência (ms)	2 903	4 421 (7218; 2859; 3187)	11 406 (16402; 9306; 8509)	5 491 (4883; 5677; 5914)
Débito no líder (reg/s)	345 (318; 331; 387)	269 (140; 351; 315)	96 (61; 108; 118)	184 (208; 176; 169)
Ligações TCP de recuperação	—	0	164 (171; 156; 166)	165 (165; 170; 161)
Registos obtidos por TCP	—	0	213 (223; 210; 207)	207 (205; 211; 206)
Registos em falta no fim (réplica 3)	—	0	7 (10; 6; 5)	12 (15; 14; 8)
Retidos em memória no fim	—	0	5 (8; 4; 3)	8 (9; 9; 7)

Nota: a COMINDIR02 tem um passo extra (produtor → PubSub → líder → PubSub → réplicas), por isso é naturalmente mais lenta do que a COMINDIR01, em que o líder lê da consola. A perda simulada de 20 % custa cerca de 2,5× mais tempo, porque cada recuperação bloqueia a thread da réplica 3 durante uma ligação TCP.

Registos em falta no fim: ficam em falta os últimos registos perdidos, porque só se recupera quando há **mais de 10 retidos**. Valores observados na réplica 3: 10, 6 e 5 (1 nó) e 15, 14 e 8 (2 nós). Os valores acima de 10 explicam-se porque o mecanismo só recupera o **primeiro intervalo em falta** de cada vez e há perdas dispersas no fim da rajada. O ficheiro nunca divergiu: foi sempre PREFIXO do líder.

6.2 Parte B (N = 3)

Teste	Resultado
(a) S01..S06	S01→p0, S02→p1, S03→p2, S04→p0, S05→p1, S06→p2. Portos 7000, 7001, 7002 à escuta. `PartitionCheck`: **OK**, cada partição em exatamente 2 réplicas, 6 registos = soma das partições.
(b) `rajada 3000 20` ×3	Total 9 000 registos: **p0=3131, p1=3214, p2=2655** (acumulado). `PartitionCheck` **OK**: todas as réplicas IGUAIS. Convergência: 17 522; 8 911; 6 888 ms.
(c) `rajada-quente 3000 20` ×3	**p0=7751, p1=655, p2=594** (acumulado): ~86 % na partição de S01. `PartitionCheck` **OK**. Convergência: 7 961; 6 691; 6 666 ms.
(d) líder p1 parado a meio	Líder p1 morto com 67 registos. O **produtor não deu erro** (pub ao Kubo continua a funcionar). p0 e p2 continuaram normalmente (1 039 e 883 registos). Das **1 078 leituras enviadas para p1**, só **67 foram registadas**: ~1 011 **perdidas sem aviso**. Após reinício, p1 retomou a numeração (`lastSeq`) e as réplicas 1 e 2 ficaram IGUAIS ao líder (88).

Teste	Resultado
(e) réplica 3 com perda 0,2; réplica 1 parada e reiniciada	Réplica 1 (reiniciada): **2 ligações**, **512 registos** via TCP; réplica 2 (sem perdas): 0 ligações; réplica 3: **299 ligações**, 370 registos via TCP. Réplicas 1 e 2 convergiram; a réplica 3 ficou com 12 em falta em p0 e em p2. Depois de mais 60 leituras: p2 convergiu, p0 ficou com 15 em falta (mesma limitação: cauda).
(f) `rajada 3000 20` ×3 com N = 1	9 000 registos numa só partição, 3 réplicas com a cópia completa (R = 3). `PartitionCheck` assinala «R = 3 (esperado 2)» porque com N = 1 há 3 cópias: não é erro, é a configuração.

Tabela de particionamento (3 execuções de `rajada 3000 20`)

Métrica	N = 1	N = 3 uniforme	N = 3 chave quente
Registos por partição (p0/p1/p2), por execução	3000 / – / –	≈1044 / 1071 / 885	≈2584 / 218 / 198
Desequilíbrio (máx. / média)	1,00	1,07	**2,58**
Débito agregado dos líderes (reg/s)	419 (384; 423; 450)	315 (172; 337; 437)	426 (380; 449; 450)
Tempo até à convergência (ms)	7 216	11 107	7 106
Registos guardados por réplica (acumulado, 9 000 no total)	9 000 / 9 000 / 9 000	6 345 / 5 869 / 5 786	8 406 / 1 249 / 8 345
Partições indisponíveis com o líder da p1 parado	1 de 1 (**100 %**)	1 de 3 (p0 e p2 continuam)	1 de 3

Leitura honesta dos números: com tudo na mesma máquina e o mesmo daemon Kubo, **N = 3 não foi mais rápido do que N = 1** (a primeira rajada de N = 3 foi particularmente lenta, 17,5 s). Os três líderes partilham o mesmo CPU, o mesmo disco e o mesmo Kubo, logo o ganho de paralelismo previsto pela teoria **não é representativo** do que se obteria com um líder por máquina. O que os testes demonstram bem é a **distribuição de dados** (cada réplica guarda só parte) e o **desequilíbrio** com a chave quente (2,58×).

6.3 Falhas introduzidas deliberadamente (ponto 14)

Falha introduzida	Lado	Exceção	Divergiu?	Recuperou?	Conclusão
Réplica parada durante a rajada e reiniciada (B-e com 3000 leituras; A-b)	réplica	nenhuma	Só como PREFIXO	**Sim**: 2 ligações, 512 registos por TCP	A recuperação cobre uma lacuna grande seguida de >10 registos novos.
`-Ddescartar` = seq da última leitura (A-c)	réplica	nenhuma	Sim: 5/6 (prefixo)	**Não** sozinha; só após 11 leituras novas	Perda da cauda não é detetada: o líder não anuncia «o último é o N» (exigiria heartbeats, fora do âmbito).
Líder com endereço anunciado errado `10.255.255.1` (F-endereco)	réplica	`SocketTimeoutException: Connect timed out` (5 s, repete em cada mensagem)	Sim: 0/60	**Não**	A propagação (PubSub) funciona, mas a recuperação exige que o endereço seja alcançável.
Porto 7000 ocupado quando o líder arranca (F-porto)	líder	`BindException: Address already in use`	—	O 2.º líder termina com código 1 (mensagem clara)	Falha rápida e visível; o 1.º líder não é afetado.
Líder fecha a ligação a meio sem `FIM` (F-semfim)	réplica	`SocketException: Connection reset`	Não no fim	**Sim, aos poucos**: cada tentativa acrescenta registos válidos; no fim 40/40 IGUAL	Sem `FIM` a resposta não é dada por completa; a réplica repete e o progresso é monótono.
`lider-p0.txt` apagado, réplicas intactas, líder reinicia (F-apagado)	líder + réplica	nenhuma	**Sim: DIVERGE**	**Não**	O líder recomeça em `seq=1`; a réplica trata 1..5 como DUPLICADO e aceita 6..15 novos: ficheiros com conteúdo diferente. O ficheiro do líder é a fonte da verdade.
Líder da partição 1 parado durante a rajada (B-d)	produtor/líder	nenhuma	Não	Parcial: após reinício retoma; **~1 011 leituras perdidas**	O produtor não sabe; troço produtor→líder sem numeração nem confirmação. p0 e p2 não são afetadas.

Falha introduzida	Lado	Exceção	Divergiu?	Recuperou?	Conclusão
Produtor com N=4 e líderes com N=3 (F-n4)	líder	`PARTIÇÃO ERRADA: S05 pertence a p1` (etc.)	—	Parcial: de 20 leituras só 7 registadas; 5 foram para `rep02-in-p3`, sem líder	A verificação no líder impede escrita na partição errada, mas as leituras rejeitadas perdem-se.
Duas instâncias de `PartitionReplica 2 1,2` na mesma pasta (F-duas)	réplica	nenhuma	**Sim: DIVERGE** (4 linhas em vez de 2)	**Não**	Cada instância tem o seu estado em memória e ambas fazem append ao mesmo ficheiro: fica corrompido. Seria preciso um lock de ficheiro/processo.

7. Tabela das construções (ponto 15)

Construção	Onde	Passou a ser garantido	Continua a não ser garantido
Endereço e porto do líder em cada `REG`	`Wire.reg`, `PartitionReplica.onMessage`	A réplica sabe sempre a quem pedir, sem configuração; acompanha mudanças do líder.	Que o endereço seja alcançável; que a mensagem seja autêntica.
`ServerSocket` / `accept()`	`PartitionLeader.main`	Há sempre um ponto de recuperação por partição.	Atende um de cada vez; líder parado = sem recuperação.
`LIMITE_RETIDOS`	`Wire`, `PartitionReplica.onMessage`	Desordem normal não dispara pedidos; memória limitada a ~10 retidos em regime normal.	Perda dos últimos registos; atraso até haver 11 retidos.
Protocolo `PEDIDO`/registos/`FIM`	líder `serve`, réplica `recover`	Fim explícito: distingue resposta completa de ligação cortada.	Autenticação; limite do tamanho do pedido.
`connect` e `setSoTimeout` com `T_TCP_MS`	`recover`	Nenhum bloqueio infinito (máx. 5 s por passo).	Que a recuperação tenha êxito; a thread da partição fica parada até 5 s.
`RecordFile.range`	líder `serve`	Resposta lida do que está persistido; sem corrida com append (`synchronized`).	Eficiência (lê o ficheiro todo); histórico perdido se o ficheiro for apagado.
`floorMod(hashCode, N)`	`Partitioner`	Mesma partição em qualquer JVM; índice sempre válido.	Boa distribuição com chave quente; estabilidade ao mudar N (65 % mudam de 3 para 4).
Sequência por partição	`PartitionLeader.seq`	Ordem total **dentro** de cada partição; líderes independentes e paralelos.	Ordem entre partições.
Atribuição de partições às réplicas (R=2)	argumentos `0,1` / `1,2` / `2,0`	Cada partição em 2 réplicas; sobrevive à falha de 1 réplica.	Falha de um líder; mudança de atribuição em funcionamento.
Secção sincronizada (`seq`+append)	`PartitionLeader.onInput`	Sem dois registos com o mesmo `seq`, nem `seq` sem registo; o servidor TCP nunca vê estado intermédio.	Falha entre append e `publish` (o registo fica no ficheiro mas não é publicado; é recuperado depois).

8. Perguntas de verificação com respostas (CA1 a CA5)

Diagnóstico inicial (secção 2.2)

P: (a) Réplica com o registo 6 recebe 8, 9, 10. O 7 perdeu-se ou ainda vem a caminho? Que critério e que custo?

R: **Não se sabe.** Pode ser entrega fora de ordem. O critério é contar: só quando há **mais de 10 retidos** se considera perdido. Custo: atraso na deteção, memória (até ~10 registos) e a perda do **último** registo nunca é detetada porque não chegam mais registos.

P: (b) No pedido TCP, o líder precisa de saber quem são as réplicas? E a réplica, quem é o líder?

R: O líder **não** precisa de saber quem são as réplicas (aceita ligações de quem vier). A réplica precisa do endereço do líder: **aprende-o nas mensagens** `REG` que recebe.

P: (c) S01 (seq 5, 10:00:01) e S02 (seq 3, 10:00:00) em partições diferentes: que conclusão?

R: **Nenhuma** sobre a ordem. Cada partição tem a sua sequência independente; os `seq` de partições diferentes não são comparáveis.

CA1 · Mecanismo de recuperação

P: Perdeu-se o registo 10. Quantos registos têm de chegar depois para a réplica o recuperar? E se for o último?

R: Têm de estar **11 registos retidos** (11, 12, …, 21), pois a condição é «mais de 10 retidos». Se o 10 for o último que o líder publica, **nunca** é recuperado: não há mais mensagens que criem retidos (testado em A-c).

P: Porque é que o endereço e o porto vão em TODAS as mensagens? E se o líder fosse reiniciado noutra máquina?

R: Porque a réplica pode arrancar a meio ou perder a primeira mensagem; assim tem sempre o endereço mais recente. Se o líder mudar de máquina, a **próxima mensagem já traz o novo endereço** e a réplica liga-se ao sítio certo. Se o anúncio fosse só na primeira mensagem, ficaria desatualizado.

P: Onde fica bloqueada a réplica durante a recuperação? O que acontece às mensagens do PubSub dessa partição?

R: Bloqueia em `connect` e depois em `readLine` (com *timeout* de 5 s), **na thread de subscrição da própria partição**. Entretanto não lê o `pubsub/sub`; as mensagens **não se perdem**, ficam em buffer até serem lidas. As outras partições têm threads próprias e continuam.

P: Porque é que «escrever localmente, depois propagar» passou a ser condição necessária?

R: O ficheiro do líder é a fonte da recuperação. Se o líder publicasse antes de escrever, uma réplica poderia pedir um registo que ainda não estaria no ficheiro. Com a ordem atual, **tudo o que foi publicado já está no ficheiro**. (O inverso, escrito mas não publicado, é inofensivo: aparece como lacuna e é recuperado.)

CA2 · Convergência

P: Réplica parada durante 500 registos. Quantas ligações TCP até convergir e quantos registos por essa via?

R: Quando volta, recebe novos `REG`, fica com tudo retido; à 11.ª mensagem retida faz **1 pedido** `último+1 .. menor retido−1` que cobre todo o intervalo parado. Medido: em B-e, 2 ligações e 512 registos; em A-e, 1 ligação e 200 registos. Ou seja, **poucas ligações** e a maior parte dos registos por TCP.

P: Durante a recuperação o líder continua a publicar. Pode um registo chegar pelas duas vias? O que impede duplicados no ficheiro?

R: Sim, pode. A regra trata os dois da mesma maneira: `seq ≤ último` ou já retido → **DUPLICADO**, ignorado. Como só se escreve `seq == último+1`, nunca se escreve duas vezes o mesmo.

P: Qual o máximo de retidos em memória em funcionamento normal? Quando deixa de se verificar?

R: **10** (ao 11.º dispara a recuperação). Deixa de se verificar quando a recuperação **falha** (líder inacessível, endereço errado): os retidos crescem sem limite até a recuperação voltar a ter êxito.

P: Porque é que LIMITE_RETIDOS não é 0? E se fosse 1000?

R: Com 0, qualquer desordem normal (um registo atrasado alguns ms) causaria uma ligação TCP desnecessária. Com 1000, cada lacuna esperaria 1000 registos antes de ser tratada (muito atraso e memória) e a perda da cauda seria ainda mais provável.

CA3 · Particionamento

P: Porque `Math.floorMod` e não `%`? Que valor dá `%` com `hashCode` negativo?

R: `%` conserva o sinal: `-7 % 3 = -1`, índice de partição inválido. `floorMod(-7,3)=2`, sempre em `[0,N)`.

P: Produtor e líder calculam a partição em processos diferentes. O que garante o mesmo resultado?

R: `String.hashCode()` tem o algoritmo fixado pela especificação do Java (igual em qualquer JVM) e `floorMod` é determinística; ambos usam o mesmo N.

P: Com uma sequência global, o que teria de existir entre os líderes? Porque está fora do âmbito?

R: Um **contador partilhado** coordenado entre líderes (consenso/relógio lógico), que cria um ponto de coordenação e anula o paralelismo. Consenso e relógios lógicos estão excluídos.

P: Que ordem é garantida entre leituras do mesmo sensor? E de partições diferentes?

R: Do mesmo sensor: **ordem do líder** (mesma partição, seq crescente). Entre partições: **nenhuma**.

CA4 · Concorrência e falhas

P: Duas atividades bloqueantes do líder; onde bloqueia cada; que estado partilham?

R: (1) thread de entrada, bloqueada em `subscribe`; (2) `main`, bloqueada em `accept()` e `readLine()`. Partilham o último `seq` e o ficheiro, protegidos por `LOCK` e pelos métodos `synchronized` de `RecordFile`.

P: Que erro se vê se `seq` e `append` não estiverem na mesma secção sincronizada? E se o servidor ler a meio de um append?

R: Duas threads podiam atribuir o **mesmo** `seq`, ou o ficheiro ficar fora de ordem. Se o servidor lesse a meio de um `append`, poderia enviar uma **linha incompleta** e a réplica falharia no `fromLine`. Por isso `append` e `range` são `synchronized`.

P: Líder da partição 1 parado: o que acontece às leituras de p1, às outras partições e às réplicas que precisam de recuperar p1?

R: Testado em B-d: as leituras de p1 **perdem-se sem erro** (~1 011 de 1 078); p0 e p2 funcionam normalmente; as réplicas 1 e 2 falham a ligação (recusada) e ficam retidas até o líder voltar.

P: Que falha da tabela não foi recuperada, e porquê?

R: Várias: cauda perdida (sem anúncio do último `seq`), endereço inalcançável, `lider-p0.txt` apagado (o histórico único desapareceu), leituras perdidas no troço produtor→líder (sem numeração), duas instâncias na mesma pasta.

CA5 · Comparação e limites

P: Custo de recuperar uma réplica parada durante uma rajada?

R: Uma ou poucas ligações TCP, com todos os registos em falta numa só resposta. Medido: 200 registos numa ligação (A-e); 512 registos em 2 ligações (B-e). Com 20 % de perda: ~164 ligações e ~213 registos por rajada de 1000.

P: Com 80 % das leituras de S01, como ficou a carga? O particionamento por chave resolve?

R: p0 recebeu ~86 % (2584 de 3000), desequilíbrio 2,58×. **Não resolve**: uma chave muito frequente cai numa só partição, e portanto num só líder.

P: N de 3 para 4: que fração muda de partição? E os registos já escritos?

R: Medido em S01..S20: **13/20 = 65 %**. Os ficheiros já escritos teriam de ser **redistribuídos**. Reparticionamento dinâmico está fora do âmbito.

P: Uma leitura perdeu-se entre produtor e líder. Quem a deteta? O que acrescentar?

R: **Ninguém**: esse troço não tem numeração. Seria preciso o produtor numerar as leituras, guardá-las até confirmação e o líder pedir/receber retransmissão; o produtor passaria a ter estado.

9. Reflexão crítica (ponto 17), respostas resumidas

P: Custo da recuperação com 20 % de perdas.

R: ~164 ligações e ~213 registos por rajada de 1000 (cerca de 1,3 registos por ligação, porque se pede apenas o primeiro buraco de cada vez). Face aos 1000 registos da propagação normal o tráfego TCP é pequeno (~21 %), mas o número de **ligações** é grande. Suportam o custo sobretudo o **líder** (aceitar e ler o ficheiro inteiro em cada pedido) e a réplica que pede (thread bloqueada). As outras réplicas não são afetadas.

P: Critério de decisão: LIMITE 1 ou 1000? Perda da cauda?

R: Com 1, a desordem normal provocaria muitos pedidos desnecessários. Com 1000, atrasos e memória enormes. A cauda não é recuperada (teste A-c) porque a decisão depende de **chegarem mais mensagens**; o líder teria de anunciar periodicamente o último `seq` (*heartbeat*), o que **não cabe** no âmbito (excluído).

P: 300 réplicas perdem o mesmo registo: ligações e tempo?

R: 300 ligações, atendidas uma a uma; a última espera ≈ 300 × t, com t = tempo de aceitar + ler o ficheiro + responder (**não medido**; com ficheiros pequenos é da ordem de alguns ms). Para atender em paralelo: uma thread por ligação aceite, dentro do âmbito. A carga do líder **continuaria a crescer** com o número de réplicas.

P: Acoplamento: líder numa rede privada/NAT/firewall.

R: Deve anunciar um endereço **alcançável pelas réplicas** (IP público/redirecionamento de porto ou endereço da VPN). Se nenhum for alcançável, a propagação PubSub continua, mas a **recuperação falha sempre** (testado com `10.255.255.1`: *connect timed out* em cada tentativa) e os retidos crescem sem limite.

P: Histórico de um milhão de registos.

R: Uma réplica nova recupera numa ligação, mas o líder lê um ficheiro de dezenas de MB em cada pedido e atende um pedido de cada vez, atrasando as outras réplicas. O tempo exato **não foi medido** (só para 200 registos); extrapolar linearmente dá uma ordem de grandeza de minutos. Evitaria percorrer tudo um índice `seq→offset` ou *snapshots*/compactação (fora do âmbito).

P: Distribuição de carga com chave quente; chave alternativa.

R: p0 ficou com ~86 %. Alternativa: chave composta `sensor + número da leitura` (ou aleatória) para espalhar S01 por várias partições; **perde-se a ordem total por sensor**.

P: Alteração de N.

R: 65 % dos sensores mudam de 3 para 4 (medido). Os ficheiros teriam de ser reorganizados. *Hashing consistente* reduz a fração (~1/N), mas está excluído.

P: Disponibilidade.

R: Na COMINDIR01 a falha do líder parava tudo. Aqui limita-se a **uma partição** (p0 e p2 continuaram em B-d), mas as leituras dessa partição perdem-se e as réplicas não conseguem recuperar. Não resolve a falha: para outra máquina assumir seria preciso eleição de líder/consenso (fora do âmbito).

P: Troço produtor → líder.

R: Em B-d perderam-se as leituras de p1 enviadas enquanto o líder estava parado e **ninguém deu conta**. Seria preciso numerar no produtor, guardar até haver confirmação e retransmitir; o produtor passaria a ter estado.

P: Segurança.

R: Qualquer processo pode publicar um `REG` com um endereço falso e levar as réplicas a ligar-se a um servidor malicioso; e quem abra ligações continuamente ao porto 7000 pode esgotar o líder (atende um de cada vez). A verificação teria de ser feita na **origem**. O campo `from` do IPFS identifica o nó emissor e ajudaria, mas todas as aplicações do mesmo nó partilham o mesmo `from`.

10. Notas finais

- **Limites cumpridos:** só biblioteca-padrão Java + `ipfs`. Sem ACKs, sem heartbeats, sem eleição de líder, sem bibliotecas externas.
- **Alteração que não vem na ficha:** repetição da publicação perante «validation ignored» (Kubo no Windows). Convém saber explicá-la se perguntarem.
- **Limitações dos testes:** tudo numa máquina; tempos muito variáveis; o que está marcado como *não medido* é estimativa.
- **Para entregar:** `git add`, commit no ramo da tarefa e Pull Request; preparar a explicação oral dos CA1–CA5 (o importante é saber justificar as escolhas, não só mostrar que os ficheiros ficaram iguais).